

KURZANLEITUNG

Franz Kneißl, Michael Niemetz

Rahmenprogramm WinAdapt für C++ Programme im Grafik-Mode

Stand 4. Juni 2022

Inhaltsverzeichnis

1 Der Programmrahmen WinAdapt	2
2 Das Beispielprojekt	3
2.1 Die unterstützten call-back-Funktionen	4
2.1.1 <code>void VtlInit(void)</code>	4
2.1.2 <code>void VtlMouse(int X, int Y)</code>	4
2.1.3 <code>void VtlKeyHit(int key)</code>	4
2.1.4 <code>void VtlPaint(int x1, int yo, int xr, int yu)</code>	4
2.1.5 <code>void VtlZyk(void)</code>	5
3 Grafik ausgeben	5
3.1 Koordinatensystem	5
3.2 Grafikfunktionen	6
3.2.1 <code>void Text(int xa, int ya, const char txt[])</code>	6
3.2.2 <code>void Line(int xa, int ya, int xe, int ye)</code>	6
3.2.3 <code>void Elli(int xa, int ya, int xe, int ye)</code>	6
3.2.4 <code>void Rect(int xa, int ya, int xe, int ye)</code>	6
3.2.5 <code>void LineCol(int r, int g, int b)</code>	6
3.2.6 <code>void FillCol(int r, int g, int b)</code>	6
3.2.7 <code>void FillPoly(int* lpPoint, int nPoints)</code>	7
3.2.8 <code>void PutPixel(int xa, int ya, COLORREF crColor)</code>	7
3.3 Konfigurationsfunktionen	7
3.3.1 <code>void setWindowTitle(const char* Title)</code>	7
3.3.2 <code>void setUpdateInterval(const unsigned Interval)</code>	7
3.4 Weitere Funktionen	7
3.4.1 <code>void delay(double amount)</code> (neu)	7
3.4.2 <code>void runAsThread(void (*Thefunction)())</code> (neu)	8
4 Listing zum Beispielprojekt	8

1 Der Programmrahmen WinAdapt

Damit ohne intime Kenntnis der Windows-Funktionalität mit Grafik gearbeitet werden kann, steht ein Adapter an das Windows API (Application Programming Interface) zur Verfügung.

Alternativ setzt der Adapter in Umgebungen mit Linux oder OS X auf GTK 2 auf.

Der Programmrahmen besteht aus einigen Quelldateien, mit denen man Grafikausgaben in ein Fenster erzeugen sowie auf Tastatur- und Mauseingaben reagieren kann. Für Grafikausgaben muss man lediglich einige Funktionen anpassen. Um den sonstigen Inhalt der vorgegebenen Dateien brauchen Sie sich nicht zu kümmern (das ist ja gerade der Zweck).

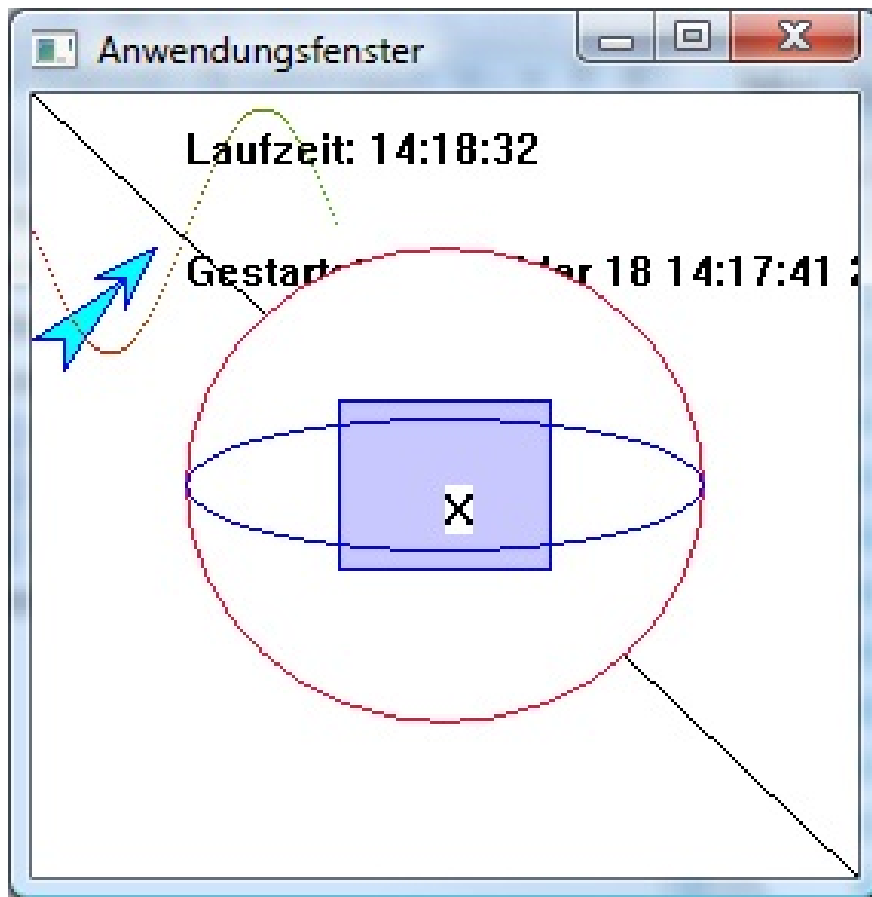


Abbildung 1: Testbild der Beispielanwendung

Das ebenfalls mitgelieferte Codeblocks-Projekt erlaubt eine Auswahl eines Windows- oder Linux-Targets und passt sich damit bezüglich der in den Übersetzungsprozess einzubindenden Systembibliotheken automatisch an die Umgebung an.

2 Das Beispielprojekt

Das Beispielprojekt zeigt, in welchen Funktionen welche Ergänzungen nötig sind, damit das Zusammenspiel mit WinAdapt klappt. Es erzeugt das in Abbildung 1 dargestellte Testbild und reagiert auf Tastatur und Maus.

Am einfachsten kommt man zu eigenen Programmen, indem man mit dem Beispielprojekt beginnt und schrittweise eigene Modifikationen einfügt und testet. Alle Funktionen, die anzupassen sind, liegen in der Quelldatei `main.cpp`, die am Ende dieser Beschreibung auf Seite 8 aufgelistet ist.

Die Funktionen in diesem Abschnitt werden vom Programmrahmen bei bestimmten Ereignissen aufgerufen (sogenannte "call-back-Funktionen"). Diese Aufrufe müssen nicht durch Ihr Programm erfolgen, sondern funktionieren automatisch.

Auf diese Weise hat man die Möglichkeit auf externe Ereignisse wie z.B. Mausklicks zu reagieren. Die entsprechende Reaktion programmiert man in die Funktion, die bei dem Ereignis aufgerufen wird.

Sie selber schreiben (bzw. modifizieren) also nicht das Hauptprogramm, sondern call-back-Funktionen, die aus WinAdapt heraus aufgerufen werden.

2.1 Die unterstützten call-back-Funktionen

2.1.1 `void Vt1Init(void)`

Wird einmal nach dem Start des Programms aufgerufen, bevor das Fenster auf dem Bildschirm erscheint. Hier können Initialisierungen wie Vorbelegungen von Variablen durchgeführt werden.

In dieser Funktion sind keine Grafikausgaben erlaubt!

2.1.2 `void Vt1Mouse(int X, int Y)`

Wird aufgerufen, wenn die linke Maustaste nach einem Klick über dem Fenster losgelassen wird. Die Parameter geben die Mausposition beim Loslassen der Taste an.

In dieser Funktion sind keine Grafikausgaben erlaubt!

2.1.3 `void Vt1KeyHit(int key)`

Wird aufgerufen, wenn ein Zeichen über die Tastatur eingegeben wird. Der Parameter gibt an, welche Taste (ASCII-Wert) gedrückt wurde.

In dieser Funktion sind keine Grafikausgaben erlaubt!

2.1.4 `void Vt1Paint(int x1, int y0, int xr, int yu)`

Wird aufgerufen, wenn Grafikausgaben nötig sind, z.B. weil das Fenster verschoben wurde oder aus sonstigen Gründen neu gezeichnet werden muss.

Dies ist die einzige Funktion, in der Grafik ausgegeben werden darf!

Das bedeutet insbesondere, dass in `Vt1Mouse`, `Vt1KeyHit` und `Vt1Zyk` nicht gezeichnet werden darf!

Die Funktion `Vt1Paint` wird vom System zyklisch oder nach Bedarf aufgerufen und muss beim nächsten Aufruf alles zeichnen, was derzeit dargestellt werden soll.

Die `Vt1Paint`-Parameter `xl` und `xr` geben die x-Werte der linken bzw. rechten Grenze des Zeichenbereichs im Fenster an, `yo` und `yu` die y-Werte der oberen bzw. unteren Grenze.

2.1.5 `void Vt1Zyk(void)`

Diese Funktion wird regelmäßig alle 200 ms aufgerufen und bietet Gelegenheit alle regelmäßig durchzuführenden Berechnungen zu erledigen.

In dieser Funktion sind keine Grafikausgaben erlaubt!

3 Grafik ausgeben

3.1 Koordinatensystem

Das verwendete Koordinatensystem ist geräteabhängig (insbesondere von der eingestellten Auflösung der Grafikkarte). Die Orientierung der y-Achse ist nach unten. Die Einheit ist Pixel.

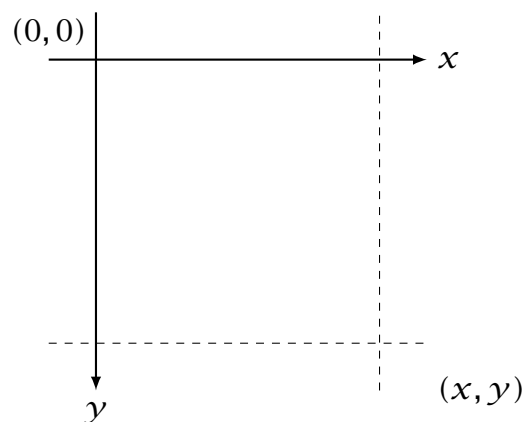


Abbildung 2: Koordinaten

Die in den folgenden Abschnitten beschriebenen Funktionen können innerhalb des Unterprogramms `Vt1Paint` aufgerufen werden. Beispielaufrufe für die Funktionen zur Ausgabe von Grafik finden Sie im Programm des Beispielprojekts. Die Funktionen sind in `WinAdapt` implementiert und stehen mit `#include <WinAdapt.h>` zur Verfügung.

3.2 Grafikfunktionen

3.2.1 `void Text(int xa, int ya, const char txt[])`

Die beiden ersten Parameter der Funktion `Text` geben die x- und y- Koordinaten des auszugebenden Textes an. Als dritter Parameter ist die Zeichenkette anzugeben.

3.2.2 `void Line(int xa, int ya, int xe, int ye)`

`Line` zeichnet eine Strecke vom Punkt, dessen x- und y- Koordinaten mit den beiden ersten Parametern gegeben sind zu dem Punkt, der durch die beiden letzten Parameter gegeben ist.

3.2.3 `void Elli(int xa, int ya, int xe, int ye)`

Wie bei `Line` werden auch beim Aufruf von `Elli` zwei Koordinatenpaare angegeben. Hier bedeuten sie den linken oberen und rechten unteren Eckpunkt des umfassenden Rechtecks der zu zeichnenden achsenparallelen Ellipse.

3.2.4 `void Rect(int xa, int ya, int xe, int ye)`

`Rect` wird ähnlich parametrisiert wie `Elli`. Gezeichnet wird das angegebene Rechteck.

3.2.5 `void LineCol(int r, int g, int b)`

Die Farbe für zu zeichnende Linien (auch der Umrandung von Flächen) wird eingestellt.

Die Funktion `LineCol` wird mit drei Argumenten aufgerufen. Die drei Parameter geben die Rot-, Grün- und Blau- Anteile der gewünschten Farbe an. Der Wertebereich der Farbanteile ist 0 (Farbe nicht enthalten) bis 255 (Farbe mit maximaler Intensität). Im Beispielpogramm wird mit `LineCol(0, 0, 255);` ein Blauton eingestellt, weswegen das nachfolgend gezeichnete Rechteck blau erscheint.

3.2.6 `void FillCol(int r, int g, int b)`

Die Funktion `FillCol` stellt die Farbe von zu zeichnenden Flächen ein. Die drei Parameter haben die gleiche Bedeutung wie bei `LineCol`.

Ist einer der Parameter negativ, wird *transparent* als Füllfarbe eingestellt, d.h. nur der Rand von Flächen gezeichnet.

3.2.7 `void FillPoly(int* lpPoint, int nPoints)`

Diese Funktion füllt einen Polygonzug. Übergeben wird ein Array von Stützstellen sowie die Anzahl der Stützstellen, wie im Beispiel gezeigt. Die Fläche wird mit der Füllfarbe für Flächen gefärbt, die Umrandung mit der Linienfarbe gezeichnet.

3.2.8 `void PutPixel(int xa, int ya, COLORREF crColor)`

Zeichnet genau einen Pixel.

Angegeben werden die Koordinaten x und y sowie die Farbe des Pixels. Die Farbe wird dabei über die Spezifikation RGB(r,g,b) angegeben, welche Farbparameter wie bei FillCol oder LineCol erwartet.

3.3 Konfigurationsfunktionen

Folgende Funktionen dürfen *ausschließlich* in `void Vt1Init(void)` aufgerufen werden, denn das Verhalten bei einem Aufruf außerhalb der Initialisierungsroutine ist undefiniert.

3.3.1 `void setWindowTitle(const char* Title)`

Definiert die Zeichenkette, welche für den Titel des Fensters verwendet wird.

3.3.2 `void setUpdateInterval(const unsigned Interval)`

Definiert das Intervall (Einheit Millisekunden) in dem die Hookfunktion Vt1Zyk aufgerufen wird. Der Defaultwert beträgt 200 ms.

3.4 Weitere Funktionen

3.4.1 `void delay(double amount)` (neu seit Juni 2022)

Verzögert die weitere Programmausführung um amount Sekunden. Die Fließkommazahl wird je nach System mit unterschiedlicher Auflösung behandelt (z.B. bei Linux: mit einer Nanosekunde, bei Windows typischerweise mit 100 Nanosekunden).

3.4.2 void runAsThread(void (*Thefunction)()) (neu seit Juni 2022)

Startet die als Funktionspointer übergebene Funktion (die keine Parameter und keinen Rückgabewert hat) in einem parallelen Thread.

Hinweis: Nachdem die gestartete Funktion nebenläufig ausgeführt wird, sind ggf. auftretend "race conditions" zu beachten und korrekt zu handhaben.

4 Listing zum Beispielprojekt

```
#include <cstdlib>
#include <stdio>
#include <cstring>
#include <cmath>
#include <ctime>
#include "WinAdapt.h"

char szStartString[128] = "Gestartet am "; // für init
char szLaufzeit[128]; // für Zyklus
int xMaus=-1, yMaus=-1; // für Maus
char nKey[] = "X"; // für KeyHit

int aPoints[] = { 40,50, 30,70, 30,60, 10,90, 10,80,
                 0,80, 30,60, 20,60, 40,50 };
int nPoints = sizeof(aPoints)/sizeof(int)/2;
int i;

void Vt1Zyk(void)
{
    time_t tLaufzeit;
    struct tm tmStruktur;
    tLaufzeit = time(0);
    tmStruktur = *localtime(&tLaufzeit);
    sprintf(szLaufzeit, "Laufzeit: %d:%d:%d",
            tmStruktur.tm_hour, tmStruktur.tm_min, tmStruktur.tm_sec);
}

void Vt1Mouse(int X, int Y)
{
    xMaus=X;
    yMaus=Y;
}
```

```

void Vt1KeyHit(int key)
{
    nKey[0] = key;
}

void Vt1Init(void)
{
    time_t tStart;
    tStart = time(0);
    strcat(szStartString, ctime(&tStart));
    setWindowTitle("Mein WinAdapt-Test");
    setUpdateInterval(200);
}

void Vt1Paint(int x1, int yo, int xr, int yu)
{
    Text(50, 5, szLaufzeit);          /* Zeichnet Text */
    Text(50, 25, szStartString);      /* Zeichnet Text */
    Line(x1, yo, xr, yu);              /* Zeichnet Linie */
    LineCol(255, 10, 50);              /* Setzt Linienfarbe */
    Elli(50, 50, xr-50, yu-50);       /* Zeichnet Ellipse */
    LineCol(0, 0, 255);                /* Setzt Linienfarbe */
    FillCol(200, 200, 255);            /* Setzt Flächenfarbe */
    Rect(100, 100, xr-100, yu-100);    /* Zeichnet Rechteck */
    FillCol(-1,-1,-1); /* Setzt Flächenfarbe transparent */
    Elli(50, 150, xr-50, yu-150);     /* Zeichnet Ellipse */

    Text((xr-x1)/2, (yu-yo)/2, nKey);

    /* Zusatzfunktionen */
    FillCol(0, 255, 255);
    FillPoly(aPoints, nPoints); /* Füllt ein Polygon */
    for(i=0; i<100; i++) /* Färbt einzelne Pixel ein */
        PutPixel(i, 45+40*sin(3.14*i/50), RGB(255-2*i, 2*i,0));

    if(xMaus!=-1 && yMaus!=-1)
    {
        FillCol(100,50,50);
        Elli(xMaus, yMaus, xMaus+7, yMaus+7);
    }
}

```